

# Support de Cours : Le Stockage de Tableaux dans localStorage

## 1. Le Problème : La limitation du localStorage

Le localStorage est un outil formidable pour sauvegarder des données dans le navigateur de l'utilisateur (qui restent même après avoir fermé l'onglet). Cependant, il a une règle d'or : il ne peut stocker que des chaînes de caractères (du texte).

Si vous essayez de stocker un tableau directement, JavaScript va le convertir maladroitement en texte, et vous perdrez la structure de votre tableau.

```
const monTableau = ["Pomme", "Banane", "Orange"];
localStorage.setItem("fruits", monTableau);
// Résultat stocké : "Pomme,Banane,Orange" -> Ce n'est plus un tableau, c'est juste du texte !
```

## 2. La Solution : Le couple JSON.stringify() et JSON.parse()

Pour contourner cette limitation, on utilise le format JSON (JavaScript Object Notation) pour transformer notre tableau en texte proprement, puis pour faire le chemin inverse.

- **Pour sauvegarder** : `JSON.stringify(monTableau)` transforme le tableau en une chaîne de caractères au format JSON.
- **Pour récupérer** : `JSON.parse(texteStocke)` retransforme la chaîne JSON en un véritable tableau JavaScript utilisable.

## 3. Guide étape par étape (Code)

Voici les trois étapes fondamentales : Écrire, Lire, et Modifier.

### Étape 1 : Sauvegarder le tableau

On transforme le tableau en texte JSON avant de l'envoyer dans le localStorage.

```
// 1. Notre tableau indexé
const utilisateurs = ["Alice", "Bob", "Charlie"];

// 2. Transformation en chaîne JSON et stockage
localStorage.setItem("listeUtilisateurs", JSON.stringify(utilisateurs));

console.log("Tableau sauvegardé !");
```

## Étape 2 : Récupérer le tableau

On extrait le texte du localStorage et on le reconvertit en tableau.

```
// 1. Récupération de la chaîne de caractères
const donneesBrutes = localStorage.getItem("listeUtilisateurs");

// 2. Conversion du texte en vrai tableau JavaScript
// (On ajoute une sécurité au cas où le localStorage serait vide)
const utilisateursRecuperes = donneesBrutes ? JSON.parse(donneesBrutes) : [];

// 3. Utilisation normale du tableau
console.log(utilisateursRecuperes); // Affiche : ["Alice", "Bob", "Charlie"]
console.log(utilisateursRecuperes[0]); // Affiche : "Alice"
```

## Étape 3 : Mettre à jour le tableau

Pour modifier un tableau déjà stocké, il faut suivre une logique en 3 temps : Lire ➡ Modifier ➡ Sauvegarder.

```
// 1. On récupère le tableau existant
const liste = JSON.parse(localStorage.getItem("listeUtilisateurs")) || [];

// 2. On effectue la modification (ex: ajouter un élément)
liste.push("David");

// 3. On écrase l'ancienne version dans le localStorage avec le nouveau tableau
localStorage.setItem("listeUtilisateurs", JSON.stringify(liste));
```

## 4. Résumé des méthodes clés

| Action       | Méthode JavaScript                 | Objectif                                        |
|--------------|------------------------------------|-------------------------------------------------|
| Sérialiser   | JSON.stringify(tableau)            | Convertit un tableau en texte pour le stockage. |
| Stocker      | localStorage.setItem("clé", texte) | Enregistre le texte dans le navigateur.         |
| Récupérer    | localStorage.getItem("clé")        | Récupère le texte stocké.                       |
| Désérialiser | JSON.parse(texte)                  | Reconvertit le texte en un vrai tableau.        |

**Supprimer**`localStorage.removeItem("clé")`

Efface la donnée de la mémoire.

⚠ **Attention aux pièges :** Si vous tentez de faire un `JSON.parse()` sur quelque chose qui n'est pas du JSON valide (ou si la clé n'existe pas et renvoie null), JavaScript générera une erreur. Pensez à toujours vérifier si la donnée existe avant de la parser (comme vu à l'étape 2).